

# Rule Engines

## Intelligent Systems II

---

Mário Antunes

Universidade de Aveiro

2026

# **Rule Engines: Bridging Logic and Automation**

---

# The Evolution of Logic

Before we can automate complex business decisions, we must understand how computers represent knowledge and truth.

- **Boolean Logic:** Relies on simple propositions and connectives like AND, OR, and NOT.
- **The Limitation:** Boolean logic is too rigid. It can state “It is raining” (True/False), but struggles to express relationships like “All sensors in room A are active.”

# First-Order Logic (FOL)

First-Order Logic (FOL) expands upon boolean logic to create a richer vocabulary for representing the world.

- **Objects:** Nouns of our logic (e.g., `sensor1`, `john`, `humidity_reading`).
- **Predicates:** Properties or relationships (e.g., `is_active(sensor1)`, `parent(john, mary)`).
- **Quantifiers:** Allows us to speak about collections (e.g.,  $\forall$  for “all”,  $\exists$  for “exists”).

To make logic computationally efficient, AI systems often restrict FOL to **Horn Clauses**.

- A Horn Clause is a disjunction of literals with at most one positive literal.
- In simpler terms, it can be written as an implication:  
$$P_1 \wedge P_2 \cdots \rightarrow Q.$$
- This specific structure enables highly efficient logical resolution, making it the foundation for logic programming.

# Logic-Based Agents

---

An **Intelligent Agent** is an entity that perceives its environment via sensors and acts upon it via actuators.

- Logic-based agents use a formal Knowledge Base (KB) to store facts and rules.
- They apply inference algorithms to this KB to deduce the best course of action.

# Agent Function vs. Agent Program

How does a logic agent know what to do?

- **Agent Function:** An abstract mathematical mapping:  
 $f : \text{Percepts} \rightarrow \text{Actions}$ .
- **Agent Program:** The concrete code that implements this function—often acting as the Inference Engine.
- *Difference:* The function defines *what* should happen, while the program determines *how* it computes it.



# Paradigms: “What” vs. “How”

Logic agents represent a fundamental shift in programming paradigms.

- **Imperative Programming:** (Python, Java) You write step-by-step instructions (“do this, then that”).
- **Declarative Programming:** (Prolog, SQL) You describe the properties of the solution (“it should be true that...”).
- In logic, you state the rules of the problem, and the engine automatically finds the solution.

# The Knowledge Base (KB)

The core of a logic agent is its Knowledge Base.

- **Facts:** Absolute truths known to the agent (e.g., `parent(john, mary)`).
- **Rules:** Conditional truths that generate new knowledge (e.g., `grandparent(X,Y) :- parent(X,Z), parent(Z,Y)`).
- Changing the agent's behavior doesn't require rewriting the core software; you simply update the facts in the KB.

# How Logic Agents Reach a Decision

To reach a decision, a logic agent uses an **Inference Engine**.

- The agent is given a **Goal** (a query).
- The engine searches through the Knowledge Base to see if the goal can be logically proven using known facts and rules.
- This is called **Resolution**.

# Search Strategies in Logic Agents

Logic agents face many competing rules. How do they navigate them?

- **Depth-First Search (DFS):** The engine follows one logical path all the way down until it finds a solution or hits a dead end.
- **Backtracking:** If a path fails, the engine “rewinds” to the last decision point and tries an alternative rule.

# Prolog

---

**Prolog** is the most popular environment for logic programming and agent creation.

- It is based strictly on Horn clause syntax.
- Instead of variables acting as memory slots (like in Python), Prolog uses variables to pattern-match and unify with facts.
- Prolog asks: *"How can I prove this goal is true?"* (Backward Chaining).

# Prolog Example

```
% A simple Knowledge Base in Prolog
```

```
parent(john, mary).
```

```
parent(mary, alice).
```

```
% A Rule
```

```
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
```

- **Query:** `?- grandparent(john, alice).`
- Prolog searches its facts, matches X to john and Y to alice, finds the intermediate Z (mary), and answers **True**.

# The Limitations of Logic Agents

While deeply powerful for symbolic math and proofs, Prolog agents have practical limits.

- **Integration Friction:** It is difficult to natively integrate Prolog with modern web APIs or microservices (often requiring bridges like PySwip).
- **Steep Learning Curve:** Developers are used to imperative logic, making strict declarative logic hard to adopt at scale.
- **Performance on Simple State:** Heavy backtracking is overkill for simple real-time IoT triggers.



# **Bridging Logic to Automation**

---

How do we keep the declarative power of logic agents but make it accessible and scalable for modern businesses?

- **The Solution:** We abstract the logic engine into a **Rule Engine**.
- We move away from strict mathematical proofs and Horn clauses towards accessible, human-readable “If-Then” business rules.

# What is a Rule Engine?

A Rule Engine is a software system that executes automated rules (“if-then” logic) over a set of facts or data.

- It successfully separates the **business logic** from the **application code**.
- It supports dynamic rule updates, complex event processing, and seamless API integration.

A standard Rule Engine consists of several core components:

1. **Rule Base:** Where the “If-Then” rules are stored.
2. **Working Memory:** Where the current facts and data (state) reside.
3. **Inference Engine:** The brain that evaluates facts against the rules.
4. **Agenda:** The manager that determines the execution order.

# The Working Memory

Unlike a Prolog Knowledge Base which holds absolute universal truths, a Rule Engine's **Working Memory** holds transient data.

- Example: An incoming JSON payload from an IoT sensor.
- It represents the *current state of the world* at the exact moment the decision needs to be made.

# Forward Chaining vs. Backward Chaining

- **Prolog (Backward Chaining):** Starts with a goal and searches backwards through rules to find supporting facts.
- **Rule Engines (Forward Chaining):** Starts with new data (facts) and searches forwards to see which rules get triggered by this new data.

# How a Decision is Reached: The Cycle

A rule engine reaches a decision through the **Match-Resolve-Act** cycle.

- Step 1: **Match**
- Step 2: **Resolve**
- Step 3: **Act**

We will break down each of these steps.

## Step 1: Match

The engine continuously compares the incoming data (facts in the Working Memory) against the conditions (when clauses) of all active rules in the Rule Base.

- If median\_humidity = 65.
- Rule 1: when: median\_humidity > 60 -> **MATCH.**
- Rule 2: when: median\_humidity <= 60 -> **NO MATCH.**



## Step 2: Resolve (Conflict Resolution)

What happens if multiple rules match the same data?

- The engine uses an **Agenda** to perform conflict resolution.
- It evaluates **Hit Policies**:
- *First Match*: Execute only the first rule that matches.
- *Collect*: Execute all matching rules and return a list of actions.
- *Priority*: Execute the rule with the highest assigned weight.

## Step 3: Act

Once the winning rule (or rules) is selected, the engine executes its consequent (then clause).

- This action might modify other data in the Working Memory.
- It might return a JSON response to an API.
- It might trigger an external system (like sending an email or turning on a physical LED).

# **Real-World Applications**

---

Rule engines are the backbone of the modern financial sector, where business policies change frequently and require instant, flawless execution.

- **Automated Loan Approvals:** Evaluating a customer's credit score, income, and debt history against constantly shifting bank criteria.
- **Dynamic Pricing:** Adjusting insurance premiums or retail prices in real-time based on live risk assessments and market facts.
- **Fraud Detection:** Instantly blocking credit card transactions that match specific, complex patterns of malicious behavior.

In healthcare, rule engines power systems that augment medical professionals by quickly cross-referencing vast amounts of medical data.

- **Clinical Expert Systems:** Providing diagnostic recommendations by evaluating a patient's current symptoms against a massive medical rule base.
- **Patient Triage:** Automatically prioritizing emergency room patients based on vital sign thresholds and incoming sensor data.
- **Treatment Protocols:** Ensuring that newly prescribed treatments do not violate safety rules, such as conflicting with a patient's known allergies.

# Real-World Applications: IoT & Regulatory Compliance

Beyond standard software, rule engines interface with the physical world and strict legal frameworks.

- **Smart IoT & Automation:** Managing complex sensor networks, such as triggering HVAC systems, factory equipment, or alarms based on sliding windows of environmental data.
- **Regulatory Compliance:** Automatically auditing business processes to ensure they meet strict, ever-changing legal regulations.
- **Agility:** Because these rules are separated from the core application code, companies can update their compliance or pricing logic the moment a new law is passed or a market shifts.

# GoRules BRMS

---

**GoRules** is a modern Business Rules Management System (BRMS) built for the cloud era.

- It replaces abstract logic code with visual editors and Decision Tables.
- It acts as an independent microservice that your applications can query via standard REST APIs.



- **Decision Tables:** Spreadsheets where columns represent inputs and outputs, and rows represent individual rules.
- **JSON Decision Model (JDM):** Under the hood, GoRules serializes all visual logic into standard JSON files.
- This allows rules to be version-controlled in Git, just like standard software code.

Why do enterprises use Rule Engines over standard Python or Java code?

- **Separation of Concerns:** Business logic is entirely decoupled from core application code.
- **Agility:** Non-developers (business analysts) can update rules dynamically without requiring a software rebuild or deployment.
- **Auditability:** It is easy to look at a decision table and understand exactly *why* a decision was made.

# Limitations of Rule Engines

However, Rule Engines are not a silver bullet.

- **Overhead:** They add architectural complexity. For a simple `if/else` script, a rule engine is massive overkill.
- **State Management:** In complex systems, tracing how facts cascade and trigger multiple unexpected rules in the Working Memory can become difficult to debug.
- **Performance:** They are generally slower than compiled imperative code due to the pattern-matching overhead.

# Advantages & Limitations of Logic Agents (Prolog)

For comparison, let's review Prolog:

- **Advantages:** Unmatched for deep, complex symbolic reasoning, recursive searches, and bidirectional proofs (a single rule can both generate and verify answers).
- **Limitations:** Very difficult to integrate into standard REST/JSON architectures, and requires a complete shift in programming mindset to use effectively.

Where are Rule Engines actively used today?

- **Internet of Things (IoT):** Smart home automation, deciding when to trigger HVAC or alarms based on sensor windows.
- **Finance:** Real-time loan approvals, dynamic credit scoring, and fraud detection.
- **Healthcare:** Expert systems that provide diagnostic recommendations based on patient symptoms.
- **Compliance:** Enforcing auditing rules and regulatory requirements automatically.

## Further Resources

---

If you want to explore further, here are some excellent resources:

- **GoRules BRMS Documentation:** [docs.gorules.io](https://docs.gorules.io)
- **Other Major Rule Engines:** [Drools \(Java\)](#), [Rulebricks](#), and [Higson](#)
- **Prolog & Logic:** [swi-prolog.org](https://swi-prolog.org)
- **Python Integration:** [PySwip](#)